

Chương 2 đã trình bày bối cảnh bài toán, các chức năng chính và các yêu cầu phi chức năng của hệ thống. Từ các yêu cầu đó, chương này trình bày các công nghệ, nền tảng lý thuyết và thư viện được sử dụng trong quá trình xây dựng hệ thống. Nội dung chương tập trung làm rõ mỗi công nghệ được dùng để giải quyết yêu cầu nào, có những lựa chọn thay thế nào và vì sao đồ án lựa chọn hướng triển khai hiện tại.

0.1 Công nghệ phát triển phía máy chủ

Phía máy chủ chịu trách nhiệm xử lý các nghiệp vụ cốt lõi của hệ thống như quản lý người dùng, khóa học, bài học, học liệu, bài kiểm tra, đơn hàng, thanh toán và quyền truy cập khóa học. Các yêu cầu ở Chương 2 cho thấy backend không chỉ cần cung cấp API cho frontend, mà còn phải bảo đảm tính nhất quán dữ liệu, kiểm soát phân quyền, xử lý tệp video bài giảng dữ liệu lớn và giữ cho các luồng nghiệp vụ có thể mở rộng về sau. Vì vậy, các công nghệ phía máy chủ được lựa chọn theo hướng rõ ràng, dễ kiểm soát và phù hợp với hệ sinh thái TypeScript.

0.1.1 Node.js, Express và TypeScript

Backend của hệ thống sử dụng Node.js làm môi trường thực thi, Express làm framework xây dựng ứng dụng web và TypeScript làm ngôn ngữ phát triển chính. Node.js phù hợp với các ứng dụng web có nhiều thao tác vào ra như truy vấn CSDL, xử lý yêu cầu HTTP, gọi dịch vụ lưu trữ, nhận thông báo thanh toán và phục vụ dữ liệu học tập **nodejs_docs**. Express cung cấp cơ chế định tuyến và middleware gọn nhẹ, giúp tổ chức các API theo từng nhóm chức năng như khóa học, bài kiểm tra, media, đơn hàng, thanh toán và người dùng **express_docs**. TypeScript bổ sung kiểu tĩnh cho JavaScript, giúp giảm lỗi khi dữ liệu đi qua nhiều lớp như controller, service, repository và mapper **typescript_docs**.

Các lựa chọn thay thế cho backend có thể là Spring Boot, Django, NestJS hoặc Fastify. Spring Boot có hệ sinh thái mạnh và phù hợp với hệ thống doanh nghiệp lớn, nhưng cần nhiều cấu hình hơn so với phạm vi đồ án. Django hỗ trợ phát triển nhanh, tuy nhiên không thống nhất ngôn ngữ với frontend. NestJS có kiến trúc đầy đủ hơn Express, nhưng cũng tạo thêm lớp trừu tượng và quy ước framework. Fastify có hiệu năng tốt, nhưng Express phổ biến hơn, tài liệu phong phú hơn và đủ đáp ứng quy mô hiện tại. Vì vậy, Express kết hợp TypeScript được lựa chọn để giữ backend gọn, dễ đọc, đồng thời vẫn có khả năng tổ chức mã nguồn theo nghiệp vụ.

0.1.2 REST API và tổ chức mã nguồn theo module

Hệ thống sử dụng REST API làm phương thức giao tiếp chính giữa frontend và backend. Các nghiệp vụ đã mô tả ở Chương 2 như xem khóa học, mua khóa học, mở bài học, làm bài kiểm tra, quản lý bài viết, quản lý đơn hàng hoặc gửi câu hỏi theo học liệu đều có thể được biểu diễn bằng các tài nguyên và endpoint rõ ràng.

Cách tiếp cận này giúp frontend gọi dữ liệu theo từng màn hình, còn backend có thể đặt các kiểm tra về xác thực, phân quyền và trạng thái nghiệp vụ ở một nơi tập trung.

Ở mức tổ chức mã nguồn, backend được chia theo các nhóm nghiệp vụ như auth, users, courses, assessments, orders, payments, media, blogs, enrollments và tutor. Trong mỗi nhóm, các phần như route, controller, service, repository, validator và mapper được tách riêng theo vai trò. Cách tổ chức này giúp người đọc dễ lần theo luồng xử lý: request đi vào route/controller, dữ liệu được kiểm tra, service xử lý nghiệp vụ, repository làm việc với CSDL, sau đó mapper chuẩn hóa dữ liệu trả về. Một lựa chọn khác là gom toàn bộ logic trong controller để triển khai nhanh hơn, nhưng cách đó dễ làm mã nguồn khó bảo trì khi số lượng chức năng tăng. GraphQL cũng có thể thay thế REST API, nhưng đồ án có nhiều luồng nghiệp vụ cần kiểm soát chặt ở backend, nên REST API phù hợp hơn về tính trực quan và khả năng kiểm thử.

0.1.3 PostgreSQL và Prisma ORM

Hệ thống sử dụng PostgreSQL làm hệ quản trị CSDL quan hệ để lưu trữ các dữ liệu có ràng buộc chặt chẽ như người dùng, vai trò, khóa học, chương, bài học, bài kiểm tra, câu hỏi, bài làm, đơn hàng, giao dịch thanh toán, quyền truy cập khóa học và tiến độ học tập. PostgreSQL hỗ trợ mô hình dữ liệu quan hệ, khóa ngoại, transaction và các cơ chế truy vấn phù hợp với yêu cầu về tính nhất quán dữ liệu đã nêu ở Mục ?? **postgresql_docs**. Đây là lựa chọn phù hợp hơn so với CSDL NoSQL trong bối cảnh hệ thống có nhiều quan hệ giữa các thực thể nghiệp vụ và cần bảo toàn dữ liệu khi xử lý thanh toán hoặc nộp bài kiểm tra.

Để thao tác với CSDL từ backend TypeScript, hệ thống sử dụng Prisma ORM. Prisma cho phép mô tả mô hình dữ liệu bằng schema, sinh client truy vấn có kiểu và hỗ trợ quản lý thay đổi lược đồ thông qua migration **prisma_docs**. So với viết SQL thuần, Prisma giảm lượng mã lặp lại trong các thao tác CRUD và giúp dữ liệu trả về có kiểu rõ ràng. So với Sequelize hoặc TypeORM, Prisma phù hợp hơn với định hướng TypeScript của đồ án vì schema và client được sinh ra nhất quán, giúp service và repository ít phụ thuộc vào chuỗi truy vấn thủ công. Trong hệ thống hiện tại, Prisma được đặt chủ yếu ở repository để phần service tập trung vào nghiệp vụ thay vì chi tiết truy vấn.

0.1.4 Zod, JWT và kiểm soát quyền truy cập

Các API của hệ thống nhận dữ liệu từ nhiều loại người dùng khác nhau, do đó backend cần kiểm tra dữ liệu đầu vào trước khi xử lý nghiệp vụ. Hệ thống sử dụng Zod để định nghĩa schema và kiểm tra dữ liệu ở runtime **zod_docs**. Cách làm này

được áp dụng cho các yêu cầu như đăng nhập, đăng ký, tạo khóa học, thiết lập bài kiểm tra, cập nhật bài học, tạo đơn hàng và hoàn tất upload học liệu. Các lựa chọn thay thế có thể là Joi hoặc Yup; tuy nhiên Zod được lựa chọn vì hỗ trợ tốt TypeScript và giúp giữ schema kiểm tra dữ liệu gần với kiểu dữ liệu mà backend sử dụng.

Về xác thực, hệ thống sử dụng JWT để biểu diễn thông tin phiên đăng nhập theo chuẩn RFC 7519 **jones2015jwt**. JWT phù hợp với ứng dụng web tách riêng frontend và backend vì frontend có thể gửi token trong các yêu cầu cần xác thực, còn backend kiểm tra token trước khi thực hiện nghiệp vụ. Về kiểm soát quyền truy cập, hệ thống sử dụng middleware kiểm tra vai trò như Học sinh, Giáo viên và Quản trị viên ở các nhóm route cần bảo vệ. Với các nghiệp vụ cần kiểm tra sâu hơn, service sử dụng thêm policy theo tài nguyên, ví dụ giáo viên chỉ được quản lý khóa học hoặc bài kiểm tra thuộc phạm vi được phép, còn quản trị viên có quyền vận hành tổng thể. Cách làm này chưa phải một mô hình phân quyền động đầy đủ, nhưng phù hợp với quy mô hiện tại và khớp với mô hình tác nhân đã trình bày trong biểu đồ use case tổng quát.

0.1.5 Transaction, số thập phân và xử lý thanh toán

Các nghiệp vụ liên quan đến đơn hàng và thanh toán cần bảo đảm rằng dữ liệu không bị cập nhật dở dang. Ví dụ, khi một giao dịch thanh toán được xác nhận, hệ thống phải ghi nhận giao dịch, cập nhật trạng thái đơn hàng và cấp quyền truy cập khóa học cho học sinh một cách nhất quán. Vì vậy, backend sử dụng transaction của CSDL cho các thao tác liên quan đến nhiều bảng dữ liệu. Nếu một bước trong nghiệp vụ thất bại, các thay đổi trước đó có thể được hủy để tránh trạng thái sai, chẳng hạn đơn hàng đã thanh toán nhưng học sinh chưa được cấp quyền học.

Đối với dữ liệu tiền tệ và điểm số, hệ thống cần tránh sai số số thực khi tính toán. Thư viện decimal.js được sử dụng để xử lý các giá trị thập phân chính xác hơn so với kiểu number thông thường của JavaScript. Đây là yêu cầu quan trọng trong các luồng như tính điểm bài kiểm tra, ghi nhận điểm tự luận, tổng hợp kết quả hoặc xử lý số tiền thanh toán. Một lựa chọn thay thế là tự quy đổi mọi số tiền về đơn vị nhỏ nhất, ví dụ đồng, rồi dùng số nguyên; cách đó phù hợp với tiền tệ nhưng không đủ linh hoạt cho điểm số có phần thập phân. Vì vậy, decimal.js được dùng cho các phép tính cần độ chính xác ổn định trong phạm vi backend.

0.1.6 Cloudflare R2 và cơ chế upload trực tiếp

Hệ thống có nhiều media cần lưu trữ như ảnh đại diện khóa học, ảnh minh họa bài viết, tài liệu PDF, tệp đính kèm và video bài giảng. Nếu các tệp này được tải trực tiếp qua backend rồi lưu trên ổ đĩa của máy chủ, backend sẽ phải

gánh băng thông lớn, khó mở rộng và khó bảo toàn dữ liệu khi triển khai trên môi trường khác. Vì vậy, hệ thống sử dụng hướng lưu trữ đối tượng cho media, trong đó backend chỉ quản lý thông tin tệp, trạng thái xử lý và quyền truy cập. Cloudflare R2 được sử dụng như dịch vụ lưu trữ đối tượng tương thích với S3 API **cloudflare_r2_docs**. Nhờ tương thích S3, backend có thể dùng AWS SDK để tạo presigned URL, kiểm tra object, tải object và xóa object bằng các thao tác của lưu trữ đối tượng **aws_s3_docs**.

Luồng upload trong hệ thống được thiết kế để frontend tải tệp trực tiếp lên R2 thông qua presigned URL. Khi người dùng chọn ảnh, tài liệu hoặc video, backend tạo bản ghi media ở trạng thái chờ upload, sinh object key theo loại tài nguyên và trả về URL upload tạm thời cho frontend. Sau khi frontend upload xong, backend kiểm tra object trên R2 để xác nhận tệp tồn tại, cập nhật kích thước, kiểu nội dung và trạng thái media. Với ảnh và tài liệu, media có thể chuyển sang trạng thái sẵn sàng. Với video, media chuyển sang trạng thái đã upload và tiếp tục đi qua bước xử lý HLS. Cách làm này giảm tải cho backend, đồng thời vẫn giữ được khả năng kiểm soát quyền sở hữu tệp và trạng thái xử lý media trong CSDL.

0.1.7 FFmpeg, FFprobe và xử lý video HLS

Video bài giảng là loại học liệu có dung lượng lớn và ảnh hưởng trực tiếp đến trải nghiệm học tập. Nếu hệ thống phát trực tiếp tệp video gốc, học sinh có thể phải tải một tệp lớn, khó tua, khó thích nghi với điều kiện mạng và khó kiểm soát trạng thái xử lý. Vì vậy, sau khi video nguồn được upload lên R2, backend sử dụng FFprobe để đọc thông tin video như thời lượng và dùng FFmpeg để chuyển video sang định dạng HLS **ffprobe_docs**, **ffmpeg_docs**. HLS chia video thành một tệp playlist và nhiều đoạn nhỏ, cho phép trình duyệt phát dần qua HTTP thay vì tải toàn bộ tệp từ đầu **pantos2017hls**.

Trong hệ thống hiện tại, video nguồn được lưu ở vùng source trên Cloudflare R2, sau đó tiến trình xử lý tải video về thư mục tạm, lấy thời lượng bằng FFprobe, chạy FFmpeg để tạo playlist `index.m3u8` và các đoạn `.ts`, rồi upload kết quả lên R2 theo thư mục riêng của media. Khi quá trình hoàn tất, backend cập nhật media sang trạng thái sẵn sàng cùng đường dẫn playlist. Nếu xử lý thất bại, media được đánh dấu lỗi để giao diện có thể thông báo thay vì hiển thị một video hỏng. Một lựa chọn thay thế là dùng dịch vụ xử lý video chuyên dụng, nhưng điều đó làm tăng chi phí và phụ thuộc nền tảng bên ngoài. Với phạm vi đồ án, FFmpeg và FFprobe cho phép chủ động xử lý video ở mức cần thiết, đủ để hỗ trợ bài giảng dạng HLS.

0.1.8 Tài liệu hóa API với OpenAPI/Swagger

Backend sử dụng tài liệu API ở dạng OpenAPI/Swagger để mô tả endpoint, request, response và các trạng thái lỗi. Tài liệu API giúp frontend và backend thống nhất hợp đồng dữ liệu, đặc biệt với các luồng có nhiều trạng thái như bài kiểm tra, upload media, thanh toán và học tập.

Một lựa chọn thay thế là chỉ viết tài liệu API thủ công trong văn bản hoặc để frontend đọc trực tiếp từ mã backend. Cách đó có thể nhanh ở giai đoạn đầu nhưng dễ sai lệch khi API thay đổi. Việc duy trì tài liệu API theo cấu trúc máy đọc được giúp quá trình kiểm tra hợp đồng FE/BE rõ ràng hơn và giảm nhầm lẫn khi hệ thống có nhiều nhóm chức năng.

0.2 Công nghệ phát triển giao diện người dùng

Frontend của hệ thống phục vụ đồng thời hai nhóm trải nghiệm chính. Nhóm thứ nhất là trải nghiệm học tập của học sinh, bao gồm tìm khóa học, mua khóa học, xem bài giảng, mở tài liệu, làm bài kiểm tra và hỏi đáp theo học liệu. Nhóm thứ hai là trải nghiệm quản trị dành cho giáo viên và quản trị viên, bao gồm tạo khóa học, sắp xếp chương bài, upload học liệu, thiết lập bài kiểm tra, chấm điểm, quản lý đơn hàng và viết blog. Vì vậy, frontend cần tổ chức giao diện theo component, quản lý dữ liệu lấy từ API, xử lý nhiều biểu mẫu phức tạp và hiển thị tốt các loại học liệu khác nhau.

0.2.1 React, Next.js và TypeScript

Hệ thống sử dụng React để xây dựng giao diện theo mô hình component. Cách tiếp cận này phù hợp với các màn hình có nhiều phần tái sử dụng như thẻ khóa học, danh sách bài học, form tạo bài kiểm tra, trình phát video, bảng quản trị và trình soạn thảo bài viết **react_docs**. Tuy nhiên, nếu chỉ dùng React theo mô hình ứng dụng một trang, hệ thống phải tự bổ sung nhiều phần như routing, cấu trúc build, tối ưu tải trang và xử lý SEO cho các trang công khai như danh sách khóa học, chi tiết khóa học hoặc bài viết blog. Vì vậy, trên nền React, hệ thống sử dụng Next.js để tổ chức route, chia nhóm trang công khai, trang học tập và trang quản trị, đồng thời hỗ trợ quá trình build và triển khai frontend **nextjs_docs**. TypeScript được dùng ở frontend để mô tả kiểu dữ liệu của form, response từ API và props của component, giúp giảm lỗi khi các màn hình trao đổi dữ liệu với nhau.

Các lựa chọn thay thế gồm Vue, Angular hoặc React kết hợp Vite. Vue có cú pháp dễ tiếp cận, Angular có framework đầy đủ nhưng nặng hơn nhu cầu của đồ án, còn React kết hợp Vite phù hợp với ứng dụng một trang gọn hơn nhưng không giải quyết sẵn các vấn đề về định tuyến theo file, tổ chức route nhiều khu vực và khả năng tối ưu các trang cần xuất hiện tốt trên công cụ tìm kiếm. Hệ thống lựa

chọn Next.js vì có cấu trúc route rõ ràng, dễ tách khu vực quản trị và khu vực học tập, đồng thời vẫn giữ được lợi thế component của React. Điều này phù hợp với yêu cầu về tính dễ bảo trì đã nêu ở Mục ??.

0.2.2 TanStack Query, Axios và Zustand

Frontend có hai loại trạng thái chính cần quản lý. Loại thứ nhất là server state, tức dữ liệu lấy từ backend như danh sách khóa học, chi tiết bài học, trạng thái media, tiến độ học tập, thông tin bài kiểm tra, đơn hàng và dữ liệu quản trị. Những dữ liệu này có vòng đời phụ thuộc vào API: đang tải, tải lỗi, tải lại, lưu cache tạm thời, hết hạn hoặc cần được cập nhật sau một thao tác ghi. Hệ thống sử dụng TanStack Query để quản lý nhóm trạng thái này, bao gồm loading, fetching, error, cache, refetch và invalidate query khi dữ liệu thay đổi **tanstack_query_docs**. Nhờ đó, các màn hình như danh sách khóa học, trang học bài hoặc trang quản trị không cần tự viết lại nhiều logic lặp về gọi API, lưu dữ liệu tạm và làm mới dữ liệu sau khi thêm, sửa hoặc xóa.

Đối với việc gửi yêu cầu HTTP, hệ thống sử dụng Axios để tạo lớp gọi API thống nhất, hỗ trợ cấu hình base URL, gửi token, xử lý lỗi và tái sử dụng logic request/response **axios_docs**. Nếu chỉ dùng Fetch API trực tiếp, mã nguồn ở các màn hình sẽ ngắn trong trường hợp đơn giản nhưng dễ lặp lại khi cần xử lý lỗi, header và trạng thái xác thực. Loại trạng thái thứ hai là client state, tức các dữ liệu cục bộ không nhất thiết phải refetch từ server theo từng màn hình, ví dụ thông tin phiên đăng nhập đã hydrate ở trình duyệt, trạng thái giỏ hàng hoặc một số lựa chọn giao diện. Những trạng thái này được quản lý bằng Zustand, một thư viện state nhỏ gọn dựa trên hook **zustand_docs**. Như vậy, TanStack Query không thay thế Zustand và Zustand cũng không dùng để cache mọi dữ liệu API; mỗi thư viện đảm nhiệm một loại trạng thái khác nhau. So với Redux, Zustand ít boilerplate hơn, dễ đọc hơn và phù hợp với quy mô hiện tại của hệ thống.

0.2.3 React Hook Form, Zod và kiểm tra dữ liệu phía giao diện

Các màn hình của hệ thống có nhiều form cần kiểm tra dữ liệu trước khi gửi lên backend, ví dụ đăng nhập, đăng ký, tạo tài khoản giáo viên, tạo khóa học, tạo chương, tạo bài học, thiết lập bài kiểm tra, tạo bài viết blog và ghi danh học sinh. Hệ thống sử dụng React Hook Form để quản lý trạng thái form và giảm số lần render không cần thiết khi người dùng nhập liệu **react_hook_form_docs**. Cách làm này giúp các form dài trong khu vực quản trị hoạt động mượt hơn và mã nguồn dễ tách thành các component nhỏ.

Zod được sử dụng cùng React Hook Form để kiểm tra dữ liệu phía frontend **zod_docs**. Việc kiểm tra dữ liệu ở frontend không thay thế kiểm tra ở backend,

nhưng giúp người dùng nhận phản hồi sớm khi nhập thiếu thông tin hoặc sai định dạng, đồng thời cũng giảm tải các request đổ lên server backend. Các lựa chọn thay thế có thể là Formik, Yup hoặc tự quản lý form bằng React state. Formik có thể dùng được nhưng thường tạo nhiều boilerplate hơn với form lớn; tự quản lý bằng state phù hợp với form nhỏ nhưng khó bảo trì khi có nhiều trường và nhiều điều kiện. Vì vậy, React Hook Form kết hợp Zod phù hợp với các màn hình nhập liệu của hệ thống.

0.2.4 Tailwind CSS và các thư viện hỗ trợ giao diện

Hệ thống sử dụng Tailwind CSS để xây dựng giao diện bằng các lớp tiện ích **tailwind_docs**. Cách tiếp cận này giúp việc tạo layout, khoảng cách, màu sắc, border, trạng thái hover và responsive được thực hiện trực tiếp trong component. Với một hệ thống có nhiều màn hình như trang học tập, trang chi tiết khóa học, bảng quản trị, form tạo bài kiểm tra và trình soạn thảo blog, Tailwind CSS giúp giao diện được phát triển nhanh mà vẫn giữ tính nhất quán.

Bên cạnh Tailwind CSS, frontend sử dụng một số thư viện nhỏ để làm giao diện gọn và rõ hơn. `clsx` và `tailwind-merge` hỗ trợ ghép class có điều kiện và xử lý xung đột class Tailwind. `lucide-react` cung cấp hệ icon nhất quán cho các nút thao tác, menu và trạng thái giao diện. `sonner` được dùng cho thông báo phản hồi như tạo thành công, lỗi upload hoặc lỗi lưu dữ liệu. `nprogress` giúp hiển thị trạng thái chuyển trang. Các thư viện này không quyết định kiến trúc hệ thống, nhưng giúp cải thiện tính dễ dùng đã nêu ở Mục ??.

0.2.5 Video.js, hls.js và trình phát bài giảng

Ở phía giao diện học tập, video bài giảng cần được phát ổn định, hỗ trợ tua, ghi nhận tiến độ và xử lý trường hợp video vẫn đang được backend xử lý. Hệ thống sử dụng Video.js trong component React để xây dựng trình phát video trên trình duyệt **videojs_docs**. Đối với nguồn HLS, hls.js được sử dụng khi trình duyệt cần thư viện hỗ trợ để đọc playlist và các đoạn video. Cách kết hợp này phù hợp với pipeline media ở backend: video được upload lên R2, chuyển sang HLS bằng FFmpeg, sau đó frontend phát playlist HLS thay vì phát trực tiếp tệp gốc.

Trình phát video không chỉ có nhiệm vụ hiển thị video. Trong hệ thống, nó còn ghi nhận thời điểm xem, trạng thái đang phát, tạm dừng và hoàn thành để phục vụ tiến độ học tập của học sinh. Khi video chưa xử lý xong hoặc gặp lỗi, giao diện cần hiển thị trạng thái rõ ràng thay vì để màn hình trống. Các lựa chọn thay thế là dùng thẻ `video` thuần của HTML hoặc nhúng video từ nền tảng bên ngoài. Thẻ `video` thuần đơn giản nhưng thiếu nhiều tiện ích khi làm việc với HLS và trạng thái người học. Nhúng video bên ngoài triển khai nhanh nhưng làm giảm khả năng

kiểm soát quyền truy cập và dữ liệu tiến độ. Vì vậy, trình phát video riêng phù hợp hơn với yêu cầu của một LMS có học liệu nội bộ.

0.2.6 Editor Tiptap xử lý rich text

Hệ thống có chức năng viết blog và quản lý nội dung học tập, trong đó giáo viên hoặc quản trị viên cần nhập văn bản có định dạng như tiêu đề, danh sách, liên kết, hình ảnh, căn lề và các đoạn nhấn mạnh. Frontend sử dụng Tiptap để xây dựng trình soạn thảo rich text trên nền React **tiptap_docs**. Tiptap phù hợp vì có kiến trúc extension, cho phép thêm các khả năng như heading, link, image, underline và text align mà không phải tự xây dựng toàn bộ editor từ đầu.

Một lựa chọn thay thế là sử dụng textarea thuần hoặc một editor dựng sẵn ít khả năng tùy biến hơn. Textarea phù hợp với nội dung văn bản đơn giản, nhưng không đáp ứng tốt nhu cầu viết bài chia sẻ kiến thức có hình ảnh, liên kết và bố cục rõ ràng. Các editor dựng sẵn có thể triển khai nhanh, tuy nhiên khó kiểm soát cấu trúc dữ liệu và các nút thao tác theo đúng giao diện quản trị của hệ thống. Tiptap được lựa chọn vì vừa đủ linh hoạt cho nhu cầu viết blog, vừa cho phép mở rộng thêm các định dạng nội dung khi hệ thống phát triển.

0.2.7 Kéo thả, thông báo và trải nghiệm quản trị

Một số màn hình quản trị của hệ thống có tính chất builder, trong đó giáo viên hoặc quản trị viên không chỉ nhập dữ liệu mà còn phải tổ chức thứ tự nội dung. Ví dụ, khi xây dựng khóa học, người dùng cần sắp xếp chương, bài học và học liệu theo đúng lộ trình học; khi tạo bài kiểm tra, người dùng cần tổ chức các câu hỏi theo thứ tự mong muốn. Nếu chỉ nhập số thứ tự thủ công, thao tác này dễ gây nhầm lẫn và không phản ánh trực tiếp cấu trúc mà người dùng đang hình dung. Vì vậy, hệ thống sử dụng @hello-pangea/dnd để hỗ trợ kéo thả trên các màn hình builder. Cách tiếp cận này giúp thao tác sắp xếp tự nhiên hơn, giảm số bước nhập liệu và làm cho trải nghiệm tạo khóa học, tạo bài kiểm tra gần hơn với cách giáo viên chuẩn bị nội dung thực tế.

Ngoài thao tác kéo thả, frontend còn sử dụng một số thư viện nhỏ để hoàn thiện trải nghiệm sử dụng. sonner được dùng để hiển thị phản hồi sau các thao tác như lưu khóa học, upload media hoặc cập nhật bài kiểm tra; nprogress hiển thị tiến trình khi chuyển trang; js-cookie hỗ trợ làm việc với cookie phía trình duyệt; isomorphic-dompurify hỗ trợ làm sạch nội dung HTML trước khi hiển thị. Những thư viện này không phải trọng tâm lý thuyết của đề án, nhưng giúp giao diện quản trị dễ dùng hơn, giảm số tình huống người dùng không biết hệ thống đã xử lý đến đâu và hạn chế việc phải tự viết lại các xử lý phổ biến ở frontend.

0.3 Nền tảng lý thuyết về RAG

Ngoài các công nghệ phát triển web, đồ án còn sử dụng nền tảng lý thuyết liên quan đến hỏi đáp theo học liệu. Chức năng này khác với các chức năng quản lý thông thường ở chỗ câu trả lời cần được tạo bằng ngôn ngữ tự nhiên, nhưng vẫn phải bám vào nội dung học liệu do giáo viên cung cấp. Vì vậy, phần này trình bày ngắn gọn về mô hình ngôn ngữ lớn và kỹ thuật RAG, làm cơ sở cho phần thiết kế và triển khai ở chương sau.

0.3.1 Mô hình ngôn ngữ lớn

Các mô hình ngôn ngữ lớn có khả năng sinh văn bản tự nhiên và trả lời câu hỏi dựa trên ngữ cảnh đầu vào. Một nền tảng quan trọng của nhiều mô hình ngôn ngữ hiện đại là kiến trúc Transformer, được đề xuất trong công trình “Attention Is All You Need” **vaswani2017attention**. Trong phạm vi đồ án, mô hình ngôn ngữ lớn được sử dụng như một dịch vụ hoặc mô hình tích hợp có sẵn, không phải đối tượng được huấn luyện lại từ đầu. Cách tiếp cận này phù hợp với đồ án ứng dụng vì mục tiêu chính là xây dựng hệ thống LMS hoàn chỉnh, không phải nghiên cứu huấn luyện mô hình mới.

Nếu chỉ gọi trực tiếp mô hình ngôn ngữ bằng câu hỏi của học sinh, câu trả lời có thể thiếu bám sát nội dung bài học hoặc chứa thông tin ngoài phạm vi học liệu. Một lựa chọn khác là chatbot theo luật cố định, nhưng cách này khó bao phủ các cách hỏi đa dạng của học sinh. Vì vậy, hệ thống cần một hướng tiếp cận kết hợp giữa khả năng sinh câu trả lời của mô hình ngôn ngữ và khả năng truy xuất nội dung liên quan từ học liệu nội bộ.

0.3.2 Retrieval-Augmented Generation

RAG là hướng tiếp cận kết hợp truy xuất thông tin và sinh ngôn ngữ. Thay vì để mô hình trả lời chỉ dựa trên tri thức đã học trong quá trình huấn luyện, hệ thống trước hết truy xuất các đoạn tài liệu liên quan, sau đó đưa các đoạn này vào ngữ cảnh để mô hình sinh câu trả lời **lewis2020rag**. Cách tiếp cận này phù hợp với yêu cầu hỏi đáp theo học liệu đã nêu ở Chương 2, vì câu trả lời cần dựa trên nội dung bài học thay vì chỉ dựa trên tri thức chung của mô hình.

Các lựa chọn thay thế cho RAG gồm fine-tuning mô hình hoặc xây dựng hệ hỏi đáp dựa trên luật. Fine-tuning yêu cầu dữ liệu huấn luyện đủ lớn, chi phí tính toán cao và quy trình đánh giá phức tạp, chưa phù hợp với phạm vi đồ án ứng dụng. Hệ hỏi đáp dựa trên luật dễ kiểm soát nhưng thiếu linh hoạt khi học sinh đặt câu hỏi theo nhiều cách khác nhau. RAG được lựa chọn vì có thể tận dụng mô hình có sẵn, bổ sung ngữ cảnh từ học liệu riêng và phù hợp với định hướng không huấn luyện mô hình từ đầu.

0.3.3 Dữ liệu ngữ cảnh và kiểm soát câu trả lời

Trong hệ thống LMS, học liệu của mỗi khóa học có phạm vi riêng. Một câu hỏi của học sinh khi đang học một bài cụ thể nên được ưu tiên trả lời dựa trên nội dung của bài đó hoặc các tài liệu liên quan trong khóa học. Vì vậy, dữ liệu ngữ cảnh của RAG cần được tổ chức theo các đơn vị học liệu như khóa học, chương, bài học, tài liệu. Khi nhận câu hỏi, hệ thống có thể tìm các đoạn liên quan, đưa vào prompt và yêu cầu mô hình trả lời trong phạm vi ngữ cảnh đã cung cấp.

Cách làm này không loại bỏ hoàn toàn rủi ro câu trả lời sai, nhưng giúp giảm tình huống mô hình trả lời xa nội dung bài học. Đồng thời, nó cũng phù hợp với định vị của đề án: chức năng hỏi đáp đóng vai trò hỗ trợ học sinh tự học và giảm các câu hỏi lặp lại cho giáo viên, không thay thế vai trò chuyên môn của giáo viên. Những chi tiết như cách tách đoạn tài liệu, lưu vector, truy xuất ngữ cảnh và thiết kế prompt sẽ được trình bày ở phần thiết kế và triển khai hệ thống.

Chương này đã trình bày các công nghệ và nền tảng lý thuyết được sử dụng để hiện thực hóa các yêu cầu đã phân tích ở Chương 2. Phía máy chủ sử dụng Node.js, Express, TypeScript, PostgreSQL, Prisma, Zod, JWT, middleware kiểm tra vai trò và các cơ chế xử lý giao dịch để xây dựng API nghiệp vụ và bảo đảm tính nhất quán dữ liệu. Các media của hệ thống được xử lý bằng Cloudflare R2, presigned URL, FFmpeg, FFprobe và HLS để hỗ trợ upload, lưu trữ và phát video bài giảng. Phía giao diện sử dụng React, Next.js, TanStack Query, Axios, Zustand, React Hook Form, Tailwind CSS, Video.js và Tiptap để xây dựng các luồng học tập và quản trị trên trình duyệt. Đối với chức năng hỏi đáp theo học liệu, chương này đã trình bày cơ sở của mô hình ngôn ngữ lớn và kỹ thuật RAG. Trên cơ sở các công nghệ đã lựa chọn, Chương 4 sẽ trình bày chi tiết quá trình phân tích, thiết kế, triển khai và đánh giá hệ thống.